Object Oriented Programming (CSEG1044)
Unit 04 – Lecture 03 Notes
Lambdas + Functional Interfaces

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **3**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Functional interfaces (concept) . . . 3
7.2 Target typing (how lambdas get a type) . . . 4
7.3 Anonymous class → lambda . . . 4
7.4 Lambda syntax (common forms) . . . 4
7.5 Common patterns . . . 4
7.5.1 Comparator for sorting . . . 4
7.5.2 Runnable for tasks/threads . . . 5
7.5.3 Callbacks/listeners . . . 5
7.6 Readability guidelines . . . 5

8 Running Project Connection **5**

9 Common Mistakes and Confusions **5**

10 Worked Example: sorting with lambda **6**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **7**

13 Self-Study Practice (no solutions) **7**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC
- **Main new ideas:** Functional interfaces (concept), Target typing (how lambdas get a type), Anonymous class → lambda
- **Course thread:** Use Lambdas + Functional Interfaces to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

2 Lecture Overview

Lambdas are a short way to write "behavior as a value" in Java. They work with **functional interfaces** (interfaces with exactly one abstract method). This is useful for:

- sorting (`Comparator`),
- callbacks and event handling (Swing listeners),
- threading (`Runnable`),
- cleaner APIs for reusable behavior.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Interfaces**: A lambda only works when a target functional interface defines exactly one abstract method. Main reference: `Unit 02 / Lecture 05 slides/notes`.
- **Anonymous classes**: Lambdas are easier to appreciate once a more verbose callback style is already known. Main reference: `Unit 03 / Lecture 01 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC

Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Identify functional interfaces and explain target typing.
- Write lambdas for common patterns (comparators, callbacks, runnable tasks).
- Apply basic readability rules for when to use lambdas.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Functional interfaces (concept)* → *Target typing (how lambdas get a type)* → *Anonymous class* → *lambda* → *Lambda syntax (common forms)* → *Common patterns*.
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Functional interface**: interface with 1 abstract method (SAM).
- **SAM**: Single Abstract Method.
- **Lambda**: expression like `(a, b) -> a + b`.
- **Target typing**: compiler infers lambda type from context (assignment/argument).
- **Effectively final**: local variable not modified after initialization (can be captured in lambda).

7 Detailed Notes

7.1 Functional interfaces (concept)

A functional interface has exactly one abstract method. It can still have:

- `default` methods,

- `static` methods.

Examples:

- `Runnable` (method `run()`)
- `Comparator<T>` (method `compare(a,b)`)
- `ActionListener` in Swing (method `actionPerformed(e)`)

Annotation (optional but useful). `@FunctionalInterface` tells the compiler to enforce the SAM rule.

7.2 Target typing (how lambdas get a type)

Lambda expressions do not have a type by themselves. They get a type from where they are used:

- assignment: `Runnable r = () -> ...`
- method argument: `button.addActionListener(e -> ...)`

If the context expects a functional interface, the lambda is converted to that interface.

7.3 Anonymous class → lambda

Think of a lambda as the short form of a one-method anonymous class.

- Anonymous class: write the interface name, the method header, and the body.
- Lambda: keep only the parameters and the body because the functional interface already gives the method name.
- Use this shortcut only when the behavior is small enough to stay readable.

7.4 Lambda syntax (common forms)

- No parameters: `() -> System.out.println("Hi")`
- One parameter: `x -> x * 2` (parentheses optional)
- Many parameters: `(a, b) -> a + b`
- Block body: `x -> { int y = x + 1; return y; }`

7.5 Common patterns

7.5.1 Comparator for sorting

Instead of writing a full comparator class, you can write:

- `(a, b) -> a.getCgpa() < b.getCgpa() ?  1 :  -1`

or use helper APIs like `Comparator.comparing`.

7.5.2 Runnable for tasks/threads

You can write a runnable task in one line and pass it to a `Thread`.

7.5.3 Callbacks/listeners

Swing event handling uses listeners, which are functional interfaces in many cases (or have adapter patterns). Lambdas make event handling code shorter.

7.6 Readability guidelines

Use lambdas when:

- the behavior is short (1–3 lines),
- the logic is clear from context,
- you do not need to reuse it elsewhere.

Prefer a named method/class when:

- the lambda becomes long/complex,
- you need to unit-test or reuse the behavior,
- error handling and branching make it hard to read.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

8 Running Project Connection

Use Lambdas + Functional Interfaces to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Assuming any interface can be used as a lambda target.

- Packing too much business logic into one lambda.
- Ignoring the target type and getting confused about the method being implemented.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

10 Worked Example: sorting with lambda

```java
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Student {
    final String rollNo;
    final double cgpa;
    Student(String rollNo, double cgpa) { this.rollNo = rollNo; this.cgpa = cgpa; }
    public double getCgpa() { return cgpa; }
    public String toString() { return rollNo + ":" + cgpa; }
}

public class Demo {
    public static void main(String[] args) {
        List<Student> list = new ArrayList<>();
        list.add(new Student("21CS1001", 8.2));
        list.add(new Student("21CS1002", 9.1));
        list.add(new Student("21CS1003", 7.6));

        list.sort((a, b) -> Double.compare(b.getCgpa(), a.getCgpa())); // desc
        System.out.println(list);

        list.sort((a, b) -> Double.compare(a.getCgpa(), b.getCgpa())); // asc
        System.out.println(list);
    }
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Functional interfaces (concept)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. What is a functional interface?

   Answer: an interface with exactly one abstract method (SAM).

2. Give one example of where you can use lambdas.

   Answer: sorting with `Comparator`, or creating a `Runnable`.

3. Can a lambda capture a local variable that changes later?

   Answer: No, captured local variables must be effectively final.

13 Self-Study Practice (no solutions)

1. Write a lambda for `Runnable` that prints numbers 1–5.
2. Sort a list of strings by length using a lambda comparator.
3. Replace a complex lambda in your code with a named method and explain why it is clearer.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. What is target typing?

Sample answer: Target typing is how the compiler infers a lambda's type from the expected functional interface in the context (assignment or method parameter).

Q2. When should you avoid using a lambda?

Sample answer: Avoid lambdas when the logic becomes long or complex, because it hurts readability and reuse. Prefer a named method/class in that case.

Q3. Why must a functional interface contain exactly one abstract method?

Sample answer: Because the lambda expression needs one unambiguous behavior contract to implement.